WHO WROTE IT?

Identifying LLMs from Their Responses

CMPSC 448 Midterm Project
Prepared by: Kirubel Tamrat (individual project)
October 11, 2026

Experiment status: The figures and performance numbers in this report were generated by the included training pipeline. Verify the raw dataset, source permissions, and findings before submission.

1. Problem and motivation

A generated answer usually arrives without an obvious signature showing which language model produced it. I wanted to see whether ordinary text patterns can serve as a useful clue. This project treats model identification as a three-class text-classification problem: given an answer, the classifier predicts whether it came from ChatGPT, Claude, or Gemini. It also tests an important possible shortcut: a classifier might learn something about the prompts instead of the models' responses.

The two required research questions are: **RQ1**, whether a CNN and an LSTM can identify a source model from its response; and **RQ2**, whether including the user's original prompt changes performance. Two additional analyses look at performance on a held-out task category (RQ3) and writing/formatting differences (RQ4).

2. Dataset and preparation

The dataset used is *LLM Response Evaluation Benchmark*, published by Pranavsakunde on GitHub (link in references). According to its public README, the dataset contains 100 shared prompts spanning general knowledge (20), coding (15), mathematics (15), writing (15), reasoning (15), translation (10), and safety (10). Each prompt was sent to ChatGPT, Claude, and Gemini; the source workbook contains their responses. This is convenient for the experiment because each source model should have the same prompt distribution. The dataset was collected by the original publisher, not by the author of this report.

The workbook was checked for empty response cells, and after validating complete prompt triplets, the pipeline retained **300 text responses** from **100 shared prompts**. Counts by model are ChatGPT: 100, Claude: 100, Gemini: 100. Prompts were split into 69 train, 16 validation, and 15 test groups. The same prompt never appears in more than one of these sets. The test partition therefore contains 45 responses, 15 per family. The uploaded workbook is the actual data input; the original author reports that these are genuine LLM-generated responses, but independent API-generation logs were not available to verify this claim.

I kept the exact text for each response, including visible formatting. The input-only experiment uses the prompt text; the output-only experiment uses the response; and the combined experiment joins both with a separator. The tokenizer splits words, numbers and punctuation into separate tokens. Its vocabulary is fitted on the training set only, so words from held-out responses are not added during training.

3. Models and training procedure

CNN: A trainable word embedding feeds three one-dimensional convolution branches, with kernel widths 3, 5 and 7. Max pooling turns each branch into a fixed-length vector, and dropout plus a fully connected layer produce three

scores. The different kernel widths are intended to capture short patterns such as repeated phrases, punctuation and formatting conventions.

LSTM: The same embedding idea feeds a bidirectional long short-term memory network. Forward and backward final hidden states are concatenated, then passed through dropout and a classification layer. Unlike the CNN, the LSTM processes tokens in sequence and can potentially use longer-range ordering.

Both classifiers are trained from scratch with cross-entropy loss and Adam (learning rate 0.002, weight decay 0.0001). The default batch size is 32, the sequence limit is 256 tokens, embeddings are 64-dimensional, and the CNN channels / LSTM hidden size are 48. Training uses up to 14 epochs, gradient clipping, and early stopping on validation loss (patience 5). The random seed is 42. The best validation checkpoint is evaluated once on the held-out test prompts. CNN filters have widths 3, 5 and 7; both models use dropout 0.3. Prompt + response is capped at the same 256-token limit as response-only, so appending a prompt may leave fewer available tokens for the response.

Reported metrics are test accuracy and macro-F1. Accuracy is easy to interpret, but macro-F1 gives equal weight to each model family. With three balanced classes, random chance is approximately 33.3%. The small dataset means the results should be treated as an initial experiment, not a reliable real-world authorship detector.

4. Experiments and results

Model	Features	Test accuracy	Macro-F1	Epochs
CNN	input	33.3%	0.314	11
CNN	output	80.0%	0.797	14
CNN	combined	75.6%	0.757	14
LSTM	input	33.3%	0.291	12
LSTM	output	55.6%	0.557	14
LSTM	combined	51.1%	0.484	11

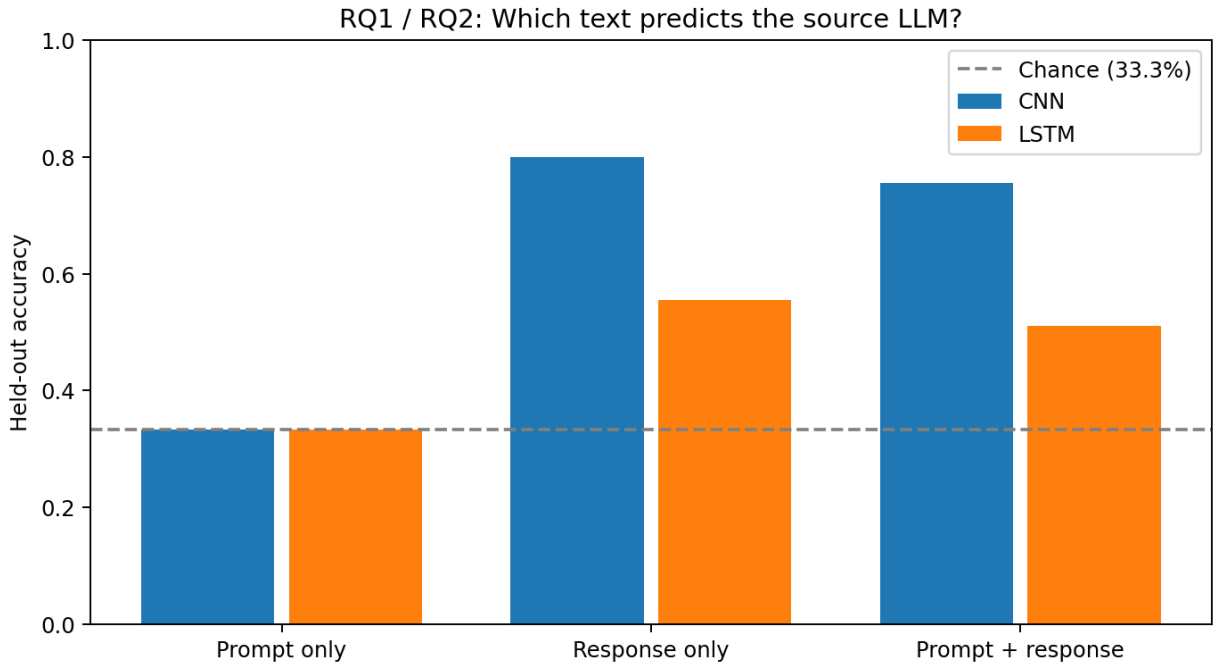


Figure 1. Same train/validation/test prompt groups are used across all six CNN and LSTM conditions.

RQ1: On response text alone, the CNN obtained 80.0% accuracy (macro-F1 0.797), while the LSTM obtained 55.6% (macro-F1 0.557). This is a performance comparison on a specific held-out split; it does not prove that either model has discovered a permanent signature of its source LLM.

RQ2, CNN: Prompt-only accuracy was 33.3%; response-only was 80.0%; and prompt plus response was 75.6%. The combined condition was 4.4 percentage points lower than response-only. Because the prompts are shared across families, a prompt-only classifier should not have a stable causal way to identify the source model.

RQ2, LSTM: Prompt-only accuracy was 33.3%; response-only was 55.6%; and prompt plus response was 51.1%. The combined condition was 4.4 percentage points lower than response-only. Because the prompts are shared across families, a prompt-only classifier should not have a stable causal way to identify the source model.

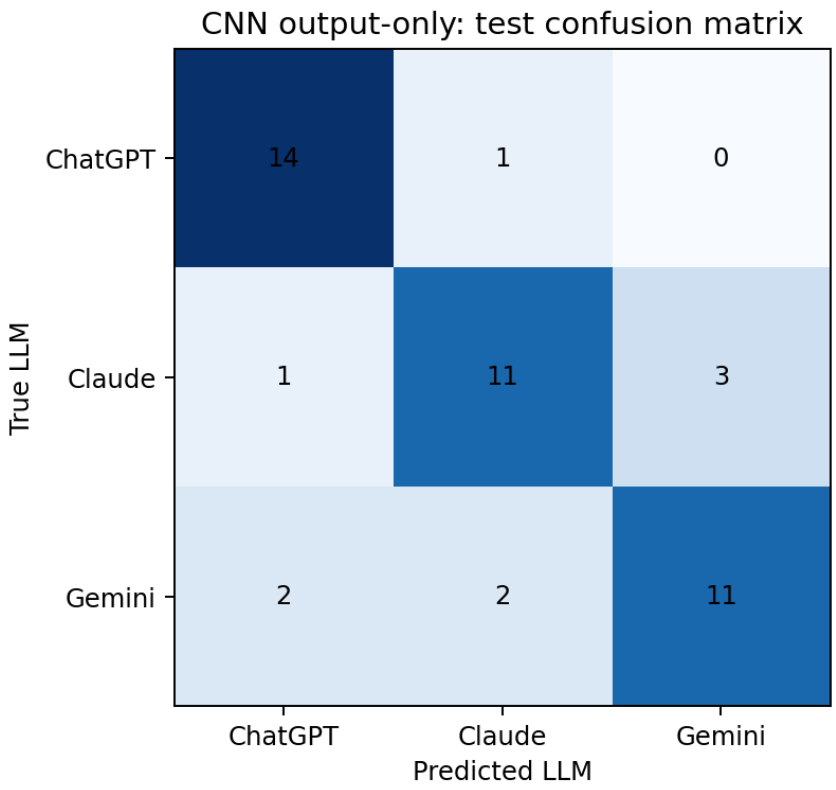


Figure 2. CNN response-only errors, broken down by true and predicted family.

5. Additional analyses

RQ3: generalization to a different kind of question. I held out the writing category completely from training and validation. Models were trained on all remaining categories, then tested on writing prompts they had never seen. This checks whether a classifier has learned patterns that travel beyond the types of questions in its training set.

Model	Held-out writing prompts	Test accuracy	Macro-F1
CNN	15	42.2%	0.338
LSTM	15	53.3%	0.524

Since the domain shift changes both question type and answer style, drops in performance would be consistent with learning category-dependent cues. Results may also be sensitive to the small number of writing prompts.

RQ4: What makes the output look different? I measured response length, average word length, lexical diversity (unique-word ratio), average sentence length, Markdown symbol count, bullet lines, and newlines. These are descriptive properties rather than proof of what the neural network learned.

Family	Words	Lexical diversity	Markdown marks	Bullets
ChatGPT	104.3	0.628	0.9	0.2
Claude	80.7	0.730	0.5	0.0
Gemini	38.5	0.813	0.1	0.0

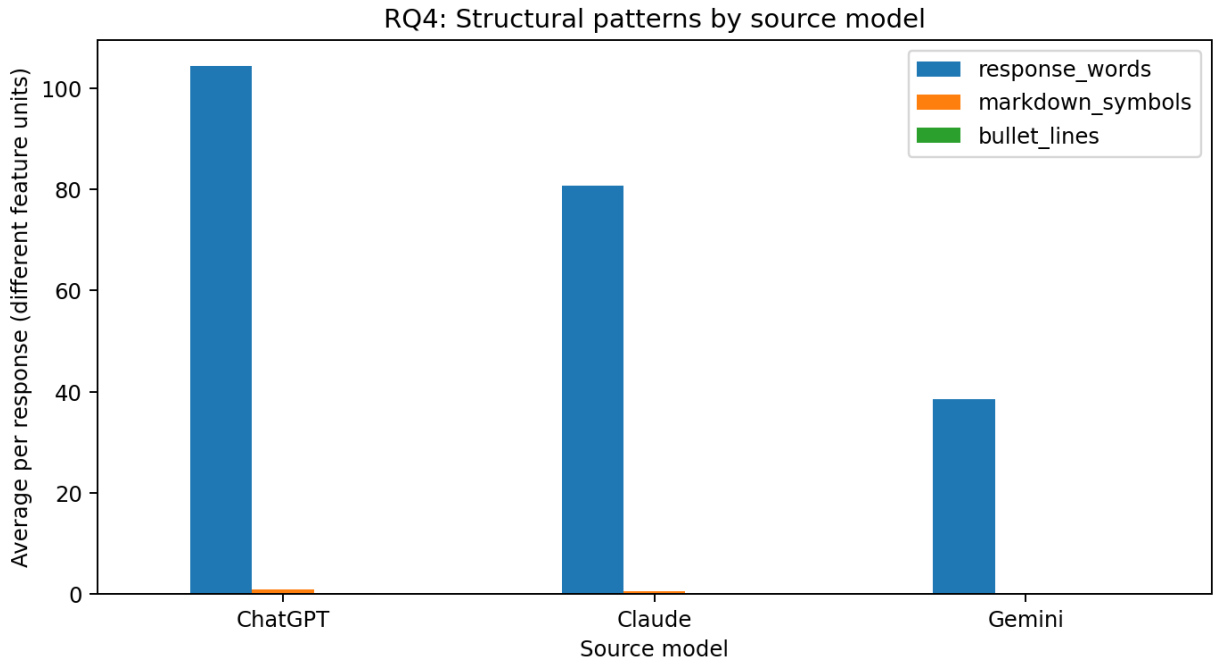


Figure 3. Average response structure. Features have different scales and should not be compared as if they share units.

To make the formatting comparison more concrete, I kept the trained response-only models fixed and stripped Markdown/list characters from the test answers. This is a *test-time perturbation*, not a new training run, so it isolates how sensitive the existing predictions are to these visible cues.

Model	Original test	Stripped test	Change
CNN	80.0%	77.8%	-2.2 pp
LSTM	55.6%	57.8%	+2.2 pp

I also trained a **simple logistic regression control** on response-style measurements. It uses the exact same prompt-disjoint train/test groups, but it does not read the text itself. This helps put the neural-network results into context: if a few numbers already identify a model, the neural networks may be using some of these simpler signals too.

Style-only control	Test accuracy	Macro-F1
Response word count	48.9%	0.446
Seven response-style features	64.4%	0.645

These controls describe predictive surface features; they do not establish that the CNN or LSTM actually relied on exactly the same properties.

6. What this experiment shows, and its limits

The biggest methodological lesson here is that dataset design matters as much as architecture. If each model had answered a different set of questions, even a prompt-only classifier might appear successful for the wrong reason. Keeping prompts shared across labels and splitting whole prompts into separate train, validation and test groups makes the experiment more defensible.

The results show that the models are not equally effective on this small corpus. The CNN performed better on response-only classification in this particular split (80.0% versus 55.6%). It would be easy to conclude that one architecture is always superior, but these are just 45 held-out responses from a single random partition. A repeated split or a larger independent benchmark would be a better way to check how stable the ranking is.

A second lesson is that an apparent 'fingerprint' may be driven by controllable features such as length, lists, or Markdown formatting. Checking descriptive text features and perturbing formatting helps investigate that possibility, although neither technique fully explains a deep-learning model's decisions.

There are several limitations: the benchmark has only 100 prompts, the response provenance depends on the public publisher's labeling, model version settings may be unavailable, and findings may not transfer to other topics, model versions, decoding settings, or edited answers. Classification of an unknown real-world paragraph should not be treated as proof of authorship. Before submitting, the student should explain the observed results in their own words and check whether these limitations match the actual run.

7. Reproducibility and source

The accompanying repository files contain preprocessing, CNN/LSTM implementations, prompt-group splitting, training and evaluation, RQ3/RQ4 analysis, figure generation, and this PDF report builder. The seed, model configuration, trained output-only checkpoints, split audit and output JSON are recorded in the results directory. Running the scripts with a verified source workbook should regenerate the numerical tables.

Dataset citation: Pranavsakunde, *LLM Response Evaluation Benchmark*, public GitHub repository: https://github.com/Pranavsakunde/LLM-Response-Evaluation-Benchmark. The public README describes 100 shared prompts with responses from ChatGPT, Gemini and Claude. Verify redistribution permissions before uploading any original response texts to GitHub.

Course specification: CMPSC 448, *Who Wrote It? Identifying LLMs from Their Responses*, modified September 17, 2026, provided assignment PDF.